Making pointer comparison non-deterministic violates the C standard. However, both GCC and LLVM (in C mode) will fold a comparison to false even when the pointers compare equal, effectively choosing code quality over standards conformance. The C semantics makes programs harder to optimize since it causes pointer comparisons to leak information about the memory layout, therefore requiring compilers to conservatively assume that most compared pointers escape (which inhibits many optimizations, such as store forwarding and dead store elimination).

A caveat of this choice of semantics is that even if a pointer comparison returns `true`, we cannot assume that two pointers have the same value. However, this was already the case because pointer comparison ignores, e.g., the extra precision fields in physical pointers like *cid*. This makes propagation of pointer equalities (e.g., GVN) unsound. We show later in this section how to make GVN for pointers correct.

For pointer inequality comparison (e.g., `p <= q`), for two logical pointers of the same block, if their offsets are inbounds the result is simply the comparison of their offsets (ICMP-PTR-ULE-LOGICAL). If the offsets are not inbounds, pointer values may overflow in hardware and hence produce a different result than if comparing the offsets. Hence, if one of the offsets is not inbounds, the result of the comparison is nondeterministic, allowing both the compiler to optimize comparisons based on the pointer offsets, as well as efficient compilation of pointer comparisons to assembly.

Comparison of logical pointers into different blocks yields a non-deterministic value. We cannot compare their integer values since that would leak information about the memory layout. We do not make the comparison yield **poison** in this case because optimizations like vectorization introduce comparisons between pointers of potentially different blocks to check at run time if vectorized accesses overlap or not (to check if it is safe to run the vectorized code).

Comparing two physical pointers is equivalent to comparing their integer representations (ICMP-PTR-PHYSICAL, ICMP-ULE-PTR-PHYSICAL). If one pointer is logical and the other physical, the logical pointer is converted to a physical pointer and then they are compared. This naturally supports the definition of comparison with pointer-integer roundtrip in the C/C++ standard: Given a valid pointer `p`, `(void*)(int)p == p` should be true. The pointer-integer round trip yields a physical pointer in our semantics, hence the comparison is always true.

4.7 Pointer Subtraction

Pointers can be subtracted to compute the difference between their offsets. This can be soundly implemented by first casting the pointers to integers and then performing an integer subtraction. In fact, this is what LLVM/Clang does today.

However, there is potential for improvement: The C/C++ standard permits an indeterminate result for the subtraction of pointers into different blocks. However, if the operation was realized as an integer subtraction, such an operation would be well defined and therefore leak information about the memory layout. Therefore, lowering pointer subtraction to subtraction of pointers casted to integers fundamentally loses precision.

In our semantics, if the program subtracts logical pointers from different blocks, the result is a **poison** value. To this end, we introduce a new instruction **psub** that takes two pointers and returns their difference. When given two logical pointers, **psub** gives the difference of their offsets if they refer to the same block (PSUB-LOGICAL). Otherwise, if the pointers refer to different logical blocks, **psub** returns **poison** (PSUB-LOGICAL-POISON). If at least one of the pointers is physical, both pointers are cast to integers and their difference is computed.

Besides the theoretical precision improvement by having a dedicated pointer subtraction instruction, there is also a practical advantage. Compiler analyses are naturally imprecise. In particular,